CI603 DATA MINING REPORT

University of Brighton

Student Number: [redacted]
Course: BSc (Hons) Computer Science with AI

Introduction

For this project a dataset from UCI has been chosen. This dataset contains 6497 red (1598) and white (4899) wines from the north of Portugal, which are each labelled with a quality score from 3 and 9 (reference). The objective of this project is to see what impact the 11 features of the wines have on the quality of the wine, finding any meaningful patterns or insights from the data. The features are as follows:

- Fixed Acidity – tartaric acid content
- Volatile Acidity – acetic acid
- Citric Acid – freshness and flavour
- Residual Sugar – remaining sugar after fermentation
- Chlorides – salt content of the wine
- Free Sulphur Dioxide
- Total Sulphur Dioxide
- Density – mass per unit volume of the wine
- pH – acidity level
- Sulphates – wine additives
- Alcohol – percentage alcohol of the wine by volume
- Quality – expert score from 3-9

All features are continuous and contain measurements in units specific to their feature. This dataset is suitable for this task as there are thousands of wines and the 11 features allow for over 70,000 individual pieces of data, making it large enough for PySpark processing.

This dataset was used in a paper by Coretz, et al., (2009) which using data mining to find wine preferences. This paper was read, analysed and used to help understand the dataset as well as how to go about data mining.

Data Preprocessing

Data Loading

The data came in two separate csv files: red and white wines. The first step was to load these two csv files and combine them. To do this an extra column was added where red wine was 0 and white was 1. The UCI paper does not combine them due to the big difference in red and white wine, but for simplicity reasons, they have been combined in this project.

Missing Values

Another reason this dataset was chosen as there are no missing values in it. Even though this was confirmed by the authors it was still necessary to check. This was done by checking for any null values in the columns.

Anomaly Detection

To identify anomalies a Z score calculation was used. This measures how far a data point is from the mean. This is calculated using: $Z = (x – \mu) / \sigma$, where X is the data point, μ is the mean and σ is the standard deviation. The Z score was calculated for all features and chlorides had the highest Z score of 15.84. For each wine the z score was calculated for chloride and any value over 3, which is an extreme outlier using z score, was remove. This flagged 107 wines but kept them for analysis as they were strange wines instead of data entry errors. Keeping these outliers is important for analysing data as these wines have extreme values which can be analysed deeper for better understanding of quality. It may be possible that these extreme values may have an impact on the quality of the wines. Also, these chloride levels do not necessarily mean low quality. After looking through the dataset, high chloride levels usually result in a medium quality wine.

Feature Standardisation

Each feature is measured via a different unit, therefore leading to a large range in scale. Without standardisation features distance-based calculations would become entirely skewed due to the difference in scale and range. Each feature was standardised with a mean of 0 and standard deviation of 1. PySpark's StandardScaler was used for this process.

Class Labelling

To allow for FP-Growth later and to reduce the imbalance in quality of wine, three classes were created: low for quality 3-4, medium for quality 5-6 and high for quality 7-9. Most of the wines lie in the medium class. A similar process was done for the features used in FP-Growth as FP-Growth only works with categorical data.

Dimensionality Reduction

Dimensionality reduction was used in this project because most of the features relate to each other or affect each other, resulting in a lot of unnecessary data being in the dataset. For example, density is largely determined by the amount of alcohol and residual sugar meaning the density feature is extra information which isn't required and the dataset won't lose information without it.

Theoretical Background

Dimensionality reduction is used to simplify data without losing too much information. This reduces noise in the dataset as well as improving efficiency. This section discusses the two related techniques: Principal Component Analysis (PCA) and Singular Value Decomposition (SVD).

Principal Component Analysis

According to Salem and Hussein (2019) "PCA is a technique that uses mathematical principles to transform a number of possibly correlated variables into a smaller number of variables called principal components." In PCA the mean values are subtracted from the dataset to centre and scale the data. Then a covariance matrix is computed to understand how the variables relate to each other. Eigenvalues and eigenvectors are identified through from this matrix. Each eigenvector defines a principal component (PC) direction, and each eigenvalue quantifies the amount of variance captured in that direction. The larger the eigenvalue the more information the corresponding eigenvector has (Wang and Zhu, 2017). Selecting the eigenvectors associated with the largest eigenvalues means dimensionality is reduced whilst retaining the maximum possible amount of information. Here is the equation for the covariance matrix:

$$var(x,y) = \frac{\Sigma(x_i - \bar{x})(y_i - \bar{y})}{n-1}$$

The eigenvalues λ and eigenvectors x are found from this matrix. PCA can also be used in financial risk modelling, image compression and natural language processing.

Singular Value Decomposition

SVD is the foundation of PCA. Any matrix $A \epsilon R^{m \times n}$ can be factorised into this equation $A = U\Sigma V^T$ where U is orthogonal $U^T U = I_n$, V is orthogonal $V^T V = I_P$ and Σ is diagonal with leading positive diagonal entries. Diagonal entries, σ, represent singular values of A and are in descending order. SVD and PCA produce the same transformations on a mean-centred matrix (Salem and Hussein, 2019). V, right singular vectors, correspond to the PC directions and the squared singular values allow for calculation of variance.

Results

After using PCA in this dataset, it reached a cumulative variance of over 90%, the threshold for this project, at 8 principal components.

```
PC 1:     31.74%   cumulative percentage:     31.74%
PC 2:     21.07%   cumulative percentage:     52.82%
PC 3:     13.01%   cumulative percentage:     65.83%
PC 4:      8.09%   cumulative percentage:     73.92%
PC 5:      6.03%   cumulative percentage:     79.95%
PC 6:      5.11%   cumulative percentage:     85.07%
PC 7:      4.47%   cumulative percentage:     89.54%
PC 8:      4.19%   cumulative percentage:     93.73%
PC 9:      2.94%   cumulative percentage:     96.66%
PC10:      2.14%   cumulative percentage:     98.80%
PC11:      0.99%   cumulative percentage:     99.80%
PC12:      0.20%   cumulative percentage:    100.00%
```

Going from 12 to 8 principal features removes redundant information whilst retaining the essential information. This allows for a clean and more efficient input for the next section

Classification

Support Vector Machine

Theoretical Background

Support Vector Machine (SVM) is a classification and regression prediction tool that uses machine learning (Jakkula, 2006). SVM perform well on high dimensional data and are resistant to overfitting making it useful for this project. The use of margin maximisation enables them to generalise to new unseen data.

It is a supervised learning algorithm which divides data into two classes using a hyperplane. SVM seeks the hyperplane that separates the classes the best, this is the margin. The points closest to the hyperplane are the support vectors. SVM can be used for things other than data mining, such as: bioinformatics, to classify proteins; financial forecasting, like stock market prediction and also continuous value prediction.

Naïve Bayes

Theoretical Background

Bayesian classifiers find the most likely of the possible classification using mathematics. Naïve bayes is a probabilistic classifier based on Bayes' Theorem:

$$P(B|A) = \frac{P(A|B)P(B)}{P(A)}$$

In data mining B is the dependent value which is the output or class and A is a set of objects with features (x1,x2,…xn) (Rish, 2001). Naïve bayes makes the simplifying assumption that all features are independent. For continuous features, which is relevant in this project, Gaussian Naïve Bayes models are used. It can be used in text classification, medical diagnosis and spam detection, it is good for these applications as it is efficient and requires minimal training data.

Classification Results

Both classifiers were used in this project for comparison purposes as they are useful for different reasons. They were trained on an 80/20 split which is common for classification algorithms.

```
SVM Results
Accuracy : 75.63%
F1 Score : 0.65
+-------------+----------+-----+
|quality_label|prediction|count|
+-------------+----------+-----+
|            0|       1.0|   50|
|            1|       1.0|  934|
|            2|       1.0|  251|
+-------------+----------+-----+

Naive Bayes Results
Accuracy : 76.03%
F1 Score : 0.74
Classifier Comparison
  Classifier   Accuracy   F1 Score
  SVM             75.63%      0.65
  Naive Bayes      76.03%      0.74
```

As we can see in the image above both produced very similar results. However, Naïve Bayes performed much better in the f1 score and slightly better in the accuracy. This is due to the fact that Naïve Bayes handled imbalances in classes better, especially under the Gaussian assumption, which has been implemented here. This can be noticed from the F1 score. It is shown by the SVM confusion matrix that the medium quality wine has the most predictions with 934 compared to the high quality's 251 and low quality's 50.

Association Analysis using FP-Growth

Theoretical Background

Frequent Pattern Growth Algorithm is an improved Apriori algorithm technique for finding frequent itemsets in large datasets. A compressed tree called the FP-Tree is constructed by reading the dataset one transaction at a time and mapping each transaction onto a path on the tree. The dataset is scanned once to determine the support count, infrequent items are removed and frequent items are sorted by most to least supported. The algorithm passes over the dataset a second time to construct the tree. The root node is represented by a null symbol initially and as paths are formed nodes get added with paths representing transactions (Sidhu, et al., 2014).

Discretisation

In the paper (Coretz, et al., 2009) that also used this dataset, bar graphs were outputted with importances attached. Sulphates were the most important for both red and white wine, and the others chosen for FP-Growth also had high importance. Each feature was labelled low, medium or high based, these values were calculated by using standard deviation, the mean and minimum and maximum values.

```
df_fp = df \
    .withColumn("alcohol",
        F.when(F.col("alcohol") < 9.5,  "alcohol_low")
         .when(F.col("alcohol") < 11.5, "alcohol_medium")
         .otherwise("alcohol_high")) \
    .withColumn("vol_acid",
        F.when(F.col("volatile acidity") < 0.3, "va_low")
         .when(F.col("volatile acidity") < 0.6, "va_medium")
         .otherwise("va_high")) \
    .withColumn("sugar",
        F.when(F.col("residual sugar") < 2.5,  "sugar_low")
         .when(F.col("residual sugar") < 10.0, "sugar_medium")
         .otherwise("sugar_high")) \
    .withColumn("sulphates",
        F.when(F.col("sulphates") < 0.5,  "sulphates_low")
         .when(F.col("sulphates") < 0.75, "sulphates_medium")
         .otherwise("sulphates_high")) \
    .withColumn("total_sulfur_dioxide",
        F.when(F.col("total sulfur dioxide") < 50,  "tsd_low")
         .when(F.col("total sulfur dioxide") < 160, "tsd_medium")
         .otherwise("tsd_high")) \
    .withColumn("quality", #quality is the consequent/target
        F.when(F.col("quality") <= 4, "quality_low")
         .when(F.col("quality") <= 6, "quality_medium")
         .otherwise("quality_high")) \
    .withColumn("wine_type", #this gives context to rules for red and white wine
        F.when(F.col("wine_type") == 0, "red_wine")
         .otherwise("white_wine"))
```

A minimum support of 10% was inputted, meaning the feature must show up at least 10% of the time in the wines. Also, a 60% confidence was used as well, meaning the rules must be correct at least 60% of the time.

I later changed this to 2% minimum support and 40% confidence so that rules would appear for high quality wine as the table would return empty with the previous values.

Results

antecedent	consequent	confidence	lift	support
[tsd_high, va_medium, quality_medium]	[white_wine]	1.0	1.3264597795018376	0.07911343697090965
[sugar_high, tsd_high, sulphates_low]	[white_wine]	1.0	1.3264597795018376	0.0417115591811605 3
[sugar_high, alcohol_low, sulphates_low, tsd_medium]	[white_wine]	1.0	1.3264597795018376	0.03340003078343851
[tsd_high, alcohol_low, sugar_medium]	[white_wine]	1.0	1.3264597795018376	0.03201477605048484
[sugar_high, alcohol_low, sulphates_low, va_low, tsd_medium]	[white_wine]	1.0	1.3264597795018376	0.024934585193166076
[tsd_high, sugar_medium, sulphates_medium, va_low]	[white_wine]	1.0	1.3264597795018376	0.025088502385716485
[tsd_high, alcohol_low]	[white_wine]	1.0	1.3264597795018376	0.09912267200246268
[quality_high, sugar_medium, sulphates_low, va_low, tsd_medium]	[white_wine]	1.0	1.3264597795018376	0.02047098660920425
[tsd_high, va_medium, sulphates_medium, alcohol_medium, quality_medium]	[white_wine]	1.0	1.3264597795018376	0.023703247652762813
[tsd_high, sugar_medium, sulphates_medium, va_low, quality_medium]	[white_wine]	1.0	1.3264597795018376	0.022317992919809144
[quality_high, alcohol_high, va_medium, sulphates_low, tsd_medium]	[white_wine]	1.0	1.3264597795018376	0.023703247652762813
[sugar_high, tsd_high, va_medium, sulphates_medium, quality_medium]	[white_wine]	1.0	1.3264597795018376	0.0218562413421579 2
[sugar_high, alcohol_low, sulphates_low]	[white_wine]	1.0	1.3264597795018376	0.06233646298291519
[tsd_low, va_medium, sugar_low, sulphates_medium, alcohol_medium, quality_medium]	[red_wine]	1.0	4.063164477798624	0.02262582730490996
[tsd_high, va_medium, alcohol_medium, quality_medium]	[white_wine]	1.0	1.3264597795018376	0.038171463752501154
[tsd_high, sugar_medium, sulphates_medium, alcohol_medium]	[white_wine]	1.0	1.3264597795018376	0.03247652762813606
[tsd_high, sugar_medium, sulphates_low, alcohol_medium]	[white_wine]	1.0	1.3264597795018376	0.021394489764506697
[tsd_high, alcohol_low, sugar_medium, sulphates_low, quality_medium]	[white_wine]	1.0	1.3264597795018376	0.021548406957057103
[va_high, tsd_low, sulphates_medium, quality_medium]	[red_wine]	1.0	4.063164477798624	0.03416961674619055
[tsd_high, sugar_medium, va_medium, alcohol_medium]	[white_wine]	1.0	1.3264597795018376	0.020009235031553024

This table returns the highest confidence scores regardless of the consequent. It shows that white wine is strongly associated the combination of high total sulphur dioxide, medium volatile acidity and medium quality, but also combinations of low sulphates, low alcohol and high sugar. Red wine shows strong connections to combinations of low total sulphur dioxide, medium volatile acidity, low sugar, medium sulphates, medium alcohol and medium quality. Due to imbalance in wine types, white wine overpowers red wine in this table. These scores are all 100%

antecedent	consequent	confidence	lift	support
[red_wine, sugar_low, sulphates_medium, alcohol_medium, tsd_medium]	[quality_medium]	0.9781021897810219	1.2775894505442902	0.020624903801754656
[alcohol_low, red_wine, tsd_medium]	[quality_medium]	0.9736842105263158	1.2718187205045182	0.022779744497460366
[red_wine, sulphates_medium, alcohol_medium, tsd_medium]	[quality_medium]	0.9710743801652892	1.2684097804450913	0.03617054024934585
[red_wine, va_medium, sulphates_medium, alcohol_medium, tsd_medium]	[quality_medium]	0.9703703703703703	1.267490208342641	0.020163152224103434
[alcohol_low, red_wine, va_medium]	[quality_medium]	0.9700598802395209	1.2670846485557232	0.024934585193166076
[alcohol_low, sulphates_medium, tsd_medium]	[quality_medium]	0.9672131147540983	1.263366225685037	0.03632445744189626
[tsd_high, alcohol_low, sugar_medium, sulphates_low, white_wine]	[quality_medium]	0.9655172413793104	1.2611510891116564	0.021548406957057103
[tsd_high, alcohol_low, sugar_medium, sulphates_low]	[quality_medium]	0.9655172413793104	1.2611510891116564	0.021548406957057103
[alcohol_low, red_wine, sulphates_medium]	[quality_medium]	0.9651741293532339	1.2607029188596623	0.02985993535477913
[alcohol_low, red_wine, sugar_low, sulphates_medium]	[quality_medium]	0.9645390070921985	1.2598733271166092	0.02093273818685547
[red_wine, sugar_medium, sulphates_medium, tsd_medium]	[quality_medium]	0.9607843137254902	1.2549689759297367	0.02262582730490996
[red_wine, va_medium, alcohol_medium, tsd_medium]	[quality_medium]	0.9583333333333334	1.2517675244605282	0.02832076342927505
[alcohol_low, sulphates_medium, tsd_medium, white_wine]	[quality_medium]	0.9568345323741008	1.2498098023390696	0.02047098660920425
[red_wine, sugar_medium, alcohol_medium, tsd_medium]	[quality_medium]	0.9562043795620438	1.2489867016515075	0.020163152224103434
[va_high, red_wine, sulphates_medium, tsd_medium]	[quality_medium]	0.9551282051282052	1.2475810110007939	0.022933661690010775
[alcohol_low, va_medium, sulphates_medium]	[quality_medium]	0.953125	1.2449644400884599	0.046944743727874404
[sugar_high, alcohol_low, va_medium]	[quality_medium]	0.9527896995708155	1.2445264732833912	0.03416961674619055
[red_wine, alcohol_medium, tsd_medium]	[quality_medium]	0.9523809523809523	1.2439925708924502	0.04925350161613052
[sugar_high, alcohol_low, va_medium, white_wine]	[quality_medium]	0.9515418502202643	1.2428965421956286	0.0332461135908881
[alcohol_low, sugar_medium, sulphates_low, va_low]	[quality_medium]	0.9512195121951219	1.2424755067816058	0.02401108203786363

This table is based on the consequent of quality. With medium quality being the most common by far this is all that is shown in the top 20 results. Medium quality wine shows to be most correlated to: red wine, medium alcohol and medium total sulphur dioxide.

antecedent	consequent	confidence	lift	support
[alcohol_high, sugar_medium, va_medium, tsd_medium, white_wine]	[quality_high]	0.6338983050847458	3.2250879311946696	0.028782515006926274
[alcohol_high, sugar_medium, va_medium, tsd_medium]	[quality_high]	0.6229508196721312	3.1693903487939203	0.029244266584577 5
[alcohol_high, sugar_medium, va_medium, white_wine]	[quality_high]	0.5958083832335329	3.0312976240158678	0.030629521317531168
[alcohol_high, sugar_medium, va_medium]	[quality_high]	0.583743842364532	2.9699167923589385	0.03647837463444667
[alcohol_high, va_medium, tsd_medium, white_wine]	[quality_high]	0.578838174273859	2.9449581975389676	0.042942896721563796
[alcohol_high, va_medium, tsd_medium]	[quality_high]	0.5632183908045977	2.8654893383378788	0.0452516546098199 2
[alcohol_high, sugar_medium, tsd_medium, white_wine]	[quality_high]	0.562	2.859290524667189	0.043250731106664615
[alcohol_high, sulphates_medium, tsd_medium, white_wine]	[quality_high]	0.5590062111801242	2.844059008643122	0.027705094659073417
[alcohol_high, sugar_medium, sulphates_medium]	[quality_high]	0.5541666666666667	2.8194368311145914	0.02047098660920425
[alcohol_high, sugar_medium, tsd_medium]	[quality_high]	0.5521235521235521	2.809042065894063	0.044020317069416656
[alcohol_high, va_medium, white_wine]	[quality_high]	0.5495327102803739	2.7958606254436877	0.0452516546098199 2
[alcohol_high, sulphates_medium, white_wine]	[quality_high]	0.5386819484240688	2.740655144018148	0.028936432199476683
[alcohol_high, va_medium, sulphates_medium]	[quality_high]	0.5384615384615384	2.7395337630263237	0.023703247652762813
[alcohol_high, sugar_medium, white_wine]	[quality_high]	0.5336879432624113	2.715247116190983 7	0.04632907495767277
[alcohol_high, sulphates_medium, tsd_medium]	[quality_high]	0.5266106442577031	2.6792399026956124	0.028936432199476683
[alcohol_high, sugar_medium]	[quality_high]	0.5225225225225225	2.6584407430139616	0.05356318300754194
[alcohol_high, va_medium]	[quality_high]	0.5223463687150838	2.657544524308457	0.05756503001385255
[alcohol_high, va_medium, sulphates_low, tsd_medium, white_wine]	[quality_high]	0.5150501672240803	2.6204235994164833	0.023703247652762813
[alcohol_high, va_medium, sulphates_low, tsd_medium]	[quality_high]	0.5150501672240803	2.6204235994164833	0.023703247652762813
[alcohol_high, sugar_medium, sulphates_low, tsd_medium, white_wine]	[quality_high]	0.514018691588785	2.6151757550918844	0.025396336770817 3

To be able to see what features correlate with high quality wine, another table was printed. This table shows results now, as mentioned before, after changing the support and confidence thresholds. We can see that high alcohol is the most important feature, showing up in every single combination. As well as high alcohol, medium sugar, medium volatile acidity, medium total sulphur dioxide and white wine. These rules all have relatively low confidence however with the highest being just over 63%.

Support and lift have been outputted in the table as these determine whether a rule is worth keeping. Support tells you how common the rule is, and lift tells you whether the connection between the features is real or a result of one of them being common. If the number for lift is over 1 it is a positive correlation, implying that the features are strongly linked. The lift in the high-quality table is very high compared to the other tables, showing these connections are very strong. The lift in the other two tables is still positive but a lot lower.

Conclusion

Overall, this report has demonstrated multiple techniques used in PySpark, for data mining to gather insightful information on the UCI Wine Quality dataset. Thorough research was conducted on the theories behind these techniques as well as in-depth analysis to discover patterns and connections between what affects the quality of these wines.

Preprocessing showed 107 extreme values of chloride and standardisation was implemented for PCA and SVM to function correctly.

PCA was successful, reducing the features from 12 to 8 PCs, whilst able to retain 93% of information, this removed any noise whilst keeping the dataset's structure.

Classification with SVM resulted in a 75.63% accuracy and 0.65 F1 score, whilst Naïve Bayes resulted in a 76.03% accuracy and 0.74 F1 score. These results could be improved with finetuning the class imbalances in the wine types and wine quality. Adding weights to these classes would allow for more accurate results.

FP-Growth showed that for high-quality wine, alcohol content is high, as well as medium sugar, medium volatile acidity, medium total sulphur dioxide and is usually white wine. It also showed combinations that are strongly correlated with white wines, red wines and medium quality wines.

References

Link to dataset: https://archive.ics.uci.edu/dataset/186/wine+quality

Coretz, P, et al., (2009) Modeling wine preferences by data mining from physicochemical properties. Available at: https://repositorium.uminho.pt/server/api/core/bitstreams/f3dd2aa2-9eda-4eba-b50e-6306a0bd0c66/content [Accessed: 17 May 2026]

Jakkula, V. (2006) Tutorial on Support Vector Machine (SVM). Available at: https://www.csd.uoc.gr/~hy439/reading/SVM/Tutorial%20on%20Support%20Vector%20Machine%20(SVM)%20.pdf [Accessed: 17 May 2026]

Rish, I. (2001) An empirical study of the naive Bayes classifier. Available at: https://staff.fmi.uvt.ro/~daniela.zaharie/dm2020/RO/Proiecte/Biblio/NaiveBayes/NaiveBayes_EmpiricalStudy.pdf [Accessed: 18 May 2026]

Salem, N and Hussein, S. (2019) Data dimensional reduction and principal components analysis. Available at: https://www.sciencedirect.com/science/article/pii/S1877050919321507 [Accessed: 17 May 2026]

Sidhu, S. et al., (2014) FP Growth Algorithm Implementation. Available at: https://d1wqtxts1xzle7.cloudfront.net/71154141/pxc3895613-libre.pdf?1633275399=&response-content-disposition=inline%3B+filename%3DFP_Growth_Algorithm_Implementation.pdf&Expires=1779117848&Signature=bknULDJIOlFenbfYD7wXMUhK5WYR1edKkCXFbvtGLJCv24hazcXaB6yKgmwVyC-BgvH-80vKRH1nmkK-aoyo0FfLPNq1~39vuiHrSTBiqtiT4flEdxkXaJybro5un7UWloAA~1OYwWKPxJtREmx7EA9dowdBOcXLuIoiXzHxbcjI5B4EQXuWt4gLQumjv6vSw1C37nXTWqjElL1Dqwvi~5HWbaKgMznkoHxawtvrOnsBlhnMHFgW43BxRP0F1zL0AmaylUt8ORVhb6EZjnw7jQmf-t58bFGkKDTM30NsBEmNyLC-878yKksOvzfnHz2iChyXgNNuCqz0i~RwTt69dw__&Key-Pair-Id=APKAJLOHF5GGSLRBV4ZA [Accessed: 18 May 2026]

Wang, Y and Zhu, L. (2017) Research and Implementation of SVD in Machine Learning. Available at: https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=7960038 [Accessed: 17 May 2026]